Dense Flow from Adaptive Correspondence

Cheng-Lin Hsieh **Stella X. Yu**
University of Michigan
{clhsieh,stellayu}@umich.edu

Abstract

Reliable optical flow requires combining evidence across pixels while preserving distinct pixel motions. Modern networks strengthen matching on regular feature grids, while the relationship between each unit's estimated motion and the pixels it represents remains implicit. We introduce *Dense flow from Adaptive CorrespondencE* (DACE), which learns which pixels should be matched together and represented by a shared motion estimate. Shared soft pixel-to-unit assignments pool contextual features, define unit-flow targets, and distribute predictions to pixels. Flow supervision learns the assignments and matching features jointly, with pixel-level grouping and correspondence over fewer units. Under the standard zero-shot protocol, DACE achieves the lowest average flow error on Sintel Final and KITTI among the compared methods. We also find that pixels with occluded or out-of-frame targets account for roughly half of a fine-grid baseline's average flow error despite occupying a small fraction of valid pixels. DACE reduces error in these regions and near motion boundaries. Design studies support improved use of context and more faithful representation of pixel motion. Project: DACE

1 Introduction

Optical flow estimates the motion of each pixel between two images. Limited texture or repeated patterns can make correspondence ambiguous, while occlusion and motion beyond the image boundary remove direct matches. Classical approaches combine information from surrounding pixels to support motion estimation [2, 3]. Yet neighboring pixels may lie on surfaces with different motions, and combining their evidence can suppress those differences [4] (Fig. 1). Reliable flow therefore requires sharing evidence to resolve ambiguity while preserving motion boundaries.

Modern flow networks have substantially improved the reliability and spatial precision of motion estimates made on regular feature grids. In widely used iterative architectures, each grid location carries a motion estimate that is refined from learned features and distributed to pixels through learned upsampling [1, 5, 6]. Advances in feature learning, correspondence search, and context aggregation improve matching performance [7–14]. Multi-scale processing and finer feature indexing improve spatial localization [15, 16], with high-resolution matching producing sharper motion boundaries [1]. Together, these advances strengthen inference over fixed-grid correspondence units.

However, regular-grid indexing alone does not determine how pixels should share evidence and motion estimates. Coarse units can span distinct motions. Finer indexing improves localization, but even at pixel resolution, motion along an edge or within a weakly textured region can remain locally ambiguous [3, 17]. Contextual features can resolve such ambiguity by combining evidence from elsewhere, including regions with different motion. The challenge is to combine evidence for a shared motion estimate that remains consistent with the pixels it represents. In current grid-based pipelines, this relationship remains implicit across feature learning and flow upsampling.

To combine evidence while preserving distinct pixel motions, we learn which pixels should be matched together and represented by a shared motion estimate. We introduce *Dense flow from Adaptive CorrespondencE* (DACE), which jointly learns correspondence units and matching features

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

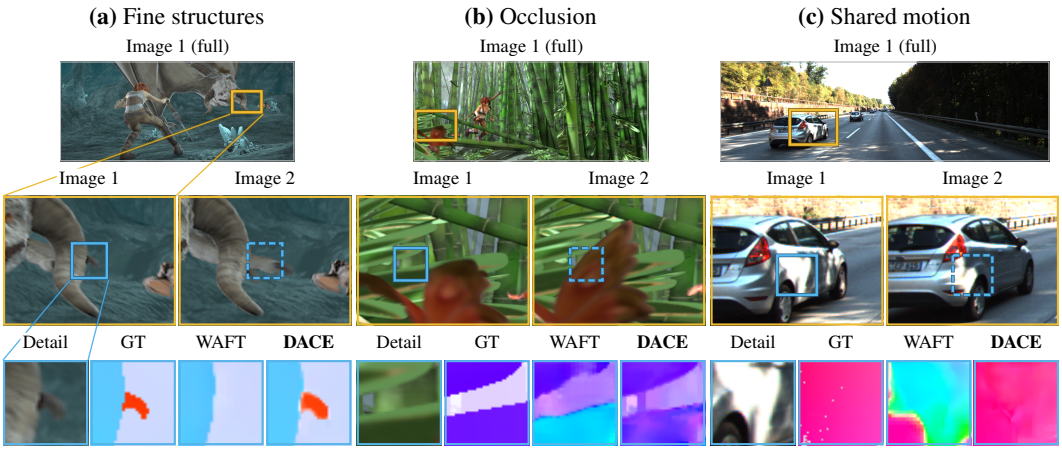


Figure 1: **Optical flow must combine compatible evidence while separating different motions.** (a) A fine structure moves differently from its surroundings. (b) A wing hides a background patch in Image 2, removing its visible match. (c) Different appearances across the car body share coherent motion. Boxes locate enlargements framed in the same color: context crops in gold, then detail and flow crops in blue. Dashed boxes repeat Image 1 coordinates in Image 2. Bottom rows compare ground-truth flow (GT) with zero-shot WAFT [1] and DACE predictions. Colors share a scale per example; gray marks unlabeled GT.

from flow supervision (Fig. 2). Each unit can draw on broader context while its estimated motion remains tied to the pixels it represents.

DACE implements this formulation through soft pixel-to-unit assignments that define each unit's support. Predicted by a compact head at image resolution, the same assignments pool features for matching, define unit-flow targets, and distribute predicted unit flow to pixels. Local context enriches the pooled features before an established iterative matching and refinement framework [1] estimates motion over fewer adaptive units. This design combines evidence for shared motion estimation while retaining pixel-level control over which estimates contribute to each pixel's flow.

Under the standard zero-shot protocol, DACE achieves the lowest average flow error on Sintel Final and KITTI among the compared published methods. Our regional analysis identifies an error concentration obscured by overall scores: occluded or out-of-frame pixels occupy only 6.4–13.9% of valid pixels yet contribute 42.3–52.2% of a fine-grid baseline's average flow error across the evaluated datasets [1]. DACE consistently achieves larger absolute error reductions in these regions and near motion boundaries than in the overall average. With fewer refinement steps, DACE retains lower error than this baseline while requiring less computation.

The learned units improve both the use of local context and the preservation of motion detail. At the same unit count, DACE outperforms fixed-grid controls and benefits more from local context. When unit motion is supplied from ground truth, its learned support and readout also retain pixel motion more faithfully. Together, these findings support learning groups for joint matching while preserving the pixel motion those groups represent.

Our main contributions are:

- We introduce adaptive correspondence units that learn which pixels to match together, linking their shared motion estimates to the same learned pixel support.
- We quantify how error in a fine-grid baseline concentrates on a small subset of pixels with occluded or out-of-frame targets, a pattern obscured by overall scores.
- We achieve the lowest zero-shot average flow error on Sintel Final and KITTI among the compared methods, with consistent improvements in regions without direct matches and near motion boundaries. Design studies support better use of context and more faithful motion representation.

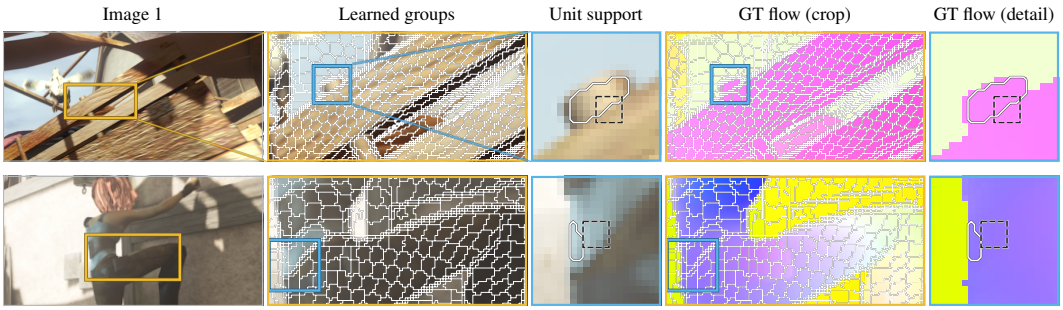


Figure 2: **DACE learns which pixels each correspondence unit represents.** Each row shows regional grouping and one selected unit from the same scene, followed by their ground-truth flow views. Neighboring groups follow slat edges and a bent body outline. The selected units follow an outer slat edge and the body contour, adapting beyond their nominal grid cells. White contours show strongest pixel assignments; dashed squares mark nominal cells, which are geometric references. Gold and blue boxes locate the two enlargement levels, framed in the same color. Each ground-truth flow (GT) view shows the same crop as its RGB counterpart. All regional groups are shown; the zero-shot model retains soft assignments.

2 Related Work

Spatial support in classical optical flow. Classical optical flow uses spatial support to infer motion from ambiguous or incomplete local measurements. Lucas–Kanade [2] pools constraints within a neighborhood, while Horn–Schunck [3] couples neighboring estimates through smoothness. Robust estimation [4] uses penalties to accommodate competing motions. Contour-based local grouping [17] makes spatial support explicit through soft affinities for local affine estimation, with motion-based boundary ownership refining these affinities. DACE learns spatial support jointly with contextual matching features from flow supervision, forming correspondence units for shared motion estimation.

Correspondence units in deep flow networks. Modern two-frame networks generally represent images on regular feature grids. Learned pyramids and cost volumes [5, 7], global matching and cost-memory encoding [8, 9], and pretrained representations [12, 13] strengthen matching evidence. Advances in recurrent estimation and learned motion priors [6, 14] improve inference. Source-image affinities aggregate motion features [10], while feature similarity guides flow propagation [8]. Multi-scale processing, learned flow upsampling, and high-resolution indexing improve spatial detail [1, 5, 15, 16]. These advances strengthen inference over fixed-grid units, while the relationship between each unit's motion estimate and the pixels it represents remains implicit across feature learning and flow upsampling. DACE learns the units used for correspondence within an established iterative matching framework [1]. Shared assignments tie unit-flow targets and dense predictions to the same pixel support, while matching features can draw on broader context.

Superpixels and differentiable grouping. Superpixel methods focus on partitioning images into compact regions that follow image boundaries [18, 19]. Learned variants make these partitions differentiable and adaptable to downstream tasks [20, 21]. DACE formulates grouping around motion prediction: each group serves as a correspondence unit that carries pixel motion. Flow supervision jointly learns the assignments and matching features, training groups both to gather useful evidence and to represent the motion of their assigned pixels.

3 Method

Overview. DACE jointly learns which pixels to group and how to match them for motion estimation. Grouping combines pixel evidence for matching, and learning to match shapes the grouping and its features. Given two images, learned assignments pool features into units, iterative matching estimates unit flow, and the same assignments distribute it back to pixels (Figure 3). Each unit can draw on broader context while estimating motion for its assigned pixels.

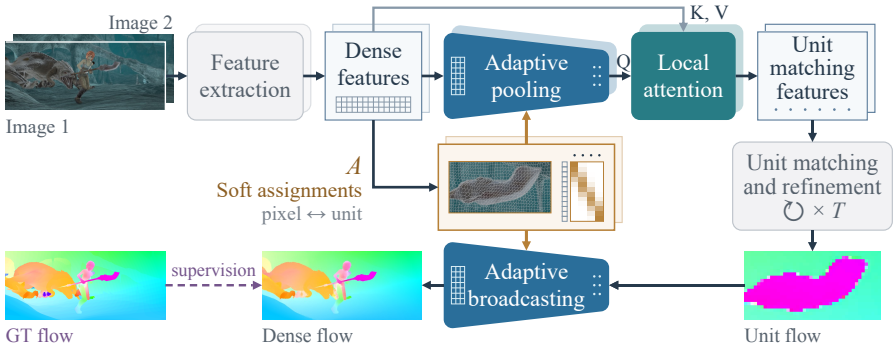


Figure 3: **DACE learns grouping and matching together.** Soft assignments A, predicted from dense image features, define adaptive units. Pooling with $P_q(A)$ forms their matching features, and broadcasting with $P_x(A)$ returns unit flow to pixels as dense flow. Image 2's branch (behind) shares weights; matching and refinement operate in unit space. The assignment and unit-flow panels enlarge the moving blade. Ground-truth flow is the only supervision (dashed); Section 3.3 details how it jointly trains the assignments, matching features and unit predictions.

3.1 Forming adaptive correspondence units

We learn each unit's pixel support to combine matching evidence while preserving distinct pixel motions.

Learning pixel-to-unit assignments. To let unit support adapt around motion boundaries and fine structures, we predict how strongly each pixel belongs to nearby units. For an image with N pixels indexed by x and M units indexed by q, the assignment weight A_{xq} connects pixel x to unit q. We collect these weights in a matrix A: each row contains one pixel's assignments and sums to one, while each column collects a unit's support.

$$A \in \mathbb{R}_+^{N \times M}, \qquad \sum_q A_{xq} = 1. \tag{1}$$

These assignments are predicted from the same dense image features used to form the units' matching features. Unit anchors provide a regular lattice for indexing, but each pixel can assign weight to the 3×3 anchors around its lattice cell. To account for their relative spatial layout, a compact head predicts a two-dimensional Gaussian over these candidate positions. Its center favors particular units, while its spread and orientation control how the pixel shares weight among them.

The head predicts five parameters: a two-dimensional mean offset from the pixel's cell anchor and three entries of a triangular factor with positive diagonal. These define the Gaussian scores, which a softmax converts to assignment weights. Each pixel has its own Gaussian, so a unit's support can be irregular and extend across cell borders.

Figure 4 connects the pixel and unit views of A, illustrating how assignments adapt near a motion boundary and share support within a coherent region.

Pooling evidence and broadcasting motion. The assignments serve two purposes: combining pixel features for unit matching, and distributing the estimated unit flow back to pixels. Pooling averages over each unit's assigned pixels; broadcasting combines unit values using each pixel's weights. Both transformations use A.

We use x and q to distinguish pixel and unit space. The operator subscript names its destination: P_q pools into unit space, and P_x broadcasts into pixel space. Let D_A be the diagonal matrix of total assignment weight per unit, with a small positive ϵ added for numerical stability. Then

$$P_q(A) = D_A^{-1} A^\top, \qquad P_x(A) = A, \qquad D_A = \mathrm{diag}(A^\top \mathbf{1}) + \epsilon I. \tag{2}$$

Both operators are determined by A; we omit the argument when it is clear. This shared relation connects feature aggregation and dense flow prediction to the same learned pixel support.

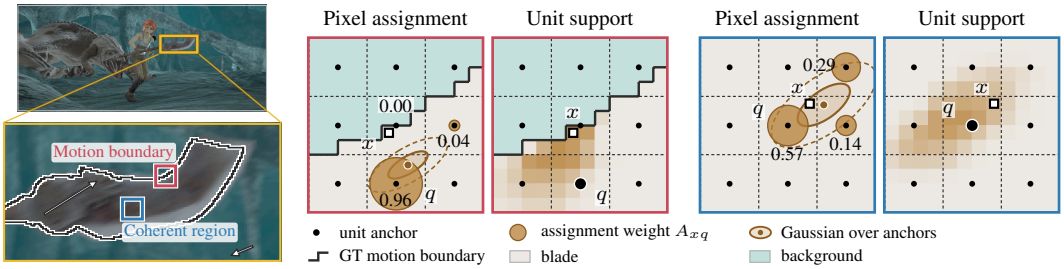


Figure 4: **Learned assignments adapt units to motion structure.** The gold box locates the moving blade in the full image; colored boxes mark two pixels x, examined in the panels framed in the same color. Pixel assignment shows the weights of x over nearby anchors, a row of A; unit support shows the column of A for its most strongly weighted unit q. Arrows show GT motion. Near the motion boundary, x favors the blade unit below over its own cell's background unit, so pixels of one cell contribute to different units; the support of q follows the boundary. In the coherent region, several units share the weight of x, giving broad, soft support.

3.2 Matching adaptive units

We estimate a unit's motion by matching the evidence gathered for its pixel group, then distribute that estimate through the same assignments.

Forming contextual matching features. Matching a unit requires evidence from its assigned pixels and context for interpreting ambiguous local observations. We first use P_q to pool dense features over the unit's learned support. The pooled features then query nearby dense features through local attention. We apply this construction to both images with shared network weights. Useful context can extend beyond the pixels whose motion the unit represents.

Estimating unit flow and returning it to pixels. We estimate unit flow from the two unit-feature maps using the iterative matching and refinement framework of WAFT [1]. Each image's assignments are predicted once and reused across updates. We write F_q for the unit flow field and F_x for the pixel flow field. Both describe forward motion from the first image to the second; their subscripts identify where the estimates are represented. Each entry is a two-dimensional flow vector.

At refinement step t, matching predicts $F_q^{(t)}$. The source image's broadcasting operator returns these estimates to its pixels:

$$F_x^{(t)} = P_x F_q^{(t)}. \tag{3}$$

The same assignments thus connect the unit's matching features with the pixels receiving its estimated flow. Because assignments participate in feature pooling, they are learned together with the matching features through flow prediction.

3.3 Joint learning of unit flow and pixel assignments

We train the units to estimate motion and the assignments to carry that motion faithfully to pixels. Figure 5 connects three objectives: predicting each group's motion, assigning pixels to compatible unit predictions, and retaining motion detail through grouping. All three use ground-truth flow for supervision. Their signatures distinguish pooling P_q and broadcasting P_x, both determined by the same assignments A. Arguments before the bar identify the supervised quantity; those after it identify information taken as given for that objective, not predictor inputs. We omit the refinement index below.

Predicting motion for each unit. Unit supervision trains each unit to predict the motion of its current pixel group. Alongside unit flow F_q, the network predicts uncertainty, following SEA-RAFT [6]. We denote these probabilistic unit-flow predictions by $\mathcal{F}_q$: each flow component has a distribution centered on its predicted value in F_q. We write $\mathrm{NLL}(\text{prediction}; \text{target})$ for the aggregated componentwise negative log-likelihood of the target under the predicted distribution.

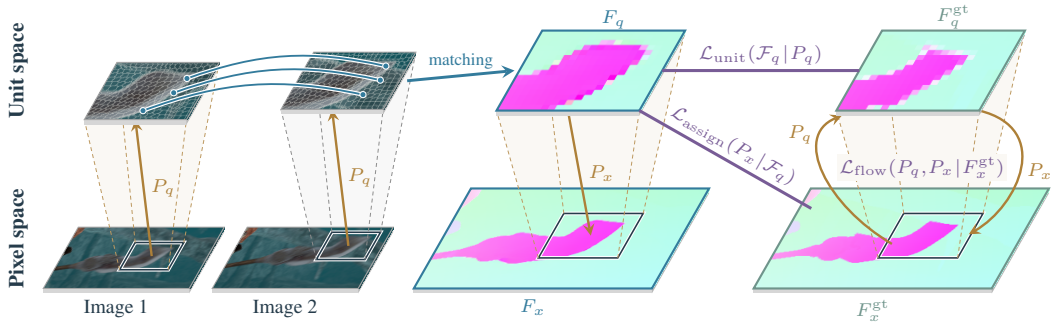


Figure 5: **Flow is estimated in unit space and supervised in both spaces.** P_q pools each image's pixels into units (colored by pooled RGB; matching uses learned features). Matching Image 1's units to Image 2's (blue; lines show three predicted correspondences) gives unit flow F_q, which P_x returns to pixels as dense flow F_x. Gold volumes mark Image 1's assignments, which also pool ground truth F_x^{gt} into unit targets F_q^{gt}. $\mathcal{L}_{\mathrm{unit}}$ fits probabilistic unit flow $\mathcal{F}_q$ to these fixed targets; $\mathcal{L}_{\mathrm{assign}}$ learns weights in P_x that select compatible neighboring unit predictions, held fixed, for each pixel of F_x^{gt}; $\mathcal{L}_{\mathrm{flow}}$ trains P_q and P_x to preserve F_x^{gt} through their round trip. Pixel views show the full crop and unit views its boxed region; flow maps share one color scale.

The superscript gt marks ground-truth flow or a target derived from it. The current pooling operator P_q gathers pixel ground truth F_x^{gt} into a motion target F_q^{gt} for each unit. Given these targets, we train the unit-flow predictions:

$$F_q^{\mathrm{gt}} = P_q F_x^{\mathrm{gt}}, \qquad \mathcal{L}_{\mathrm{unit}}(\mathcal{F}_q \mid P_q) = \mathrm{NLL}(\mathcal{F}_q; F_q^{\mathrm{gt}}). \tag{4}$$

The pooled targets are held fixed for this objective. Assignments remain trainable through feature pooling and motion prediction, so learning to match also improves the grouping.

Assigning pixels to compatible unit predictions. Assignment supervision learns broadcasting weights that select unit predictions compatible with each pixel's ground-truth motion. An accurate average unit flow can still disagree with an individual pixel near a motion boundary. Given the current probabilistic unit-flow predictions, we learn the weights in P_x.

Let $\mathrm{Mix}_{P_x}[\mathcal{F}_q]$ denote the pixelwise mixture of neighboring flow distributions, with weights derived from P_x. We minimize

$$\mathcal{L}_{\mathrm{assign}}(P_x \mid \mathcal{F}_q) = \mathrm{NLL}\big(\mathrm{Mix}_{P_x}[\mathcal{F}_q]; F_x^{\mathrm{gt}}\big)\,. \tag{5}$$

Both unit flow and its uncertainty are held fixed for this objective, while the assignments and their shared features learn. The mixture scores each pixel's label against neighboring flow predictions; it is not an error on their averaged flow.

Preserving motion through the grouping. To train grouping to retain motion differences independently of matching errors, we pool ground-truth pixel flow with P_q and broadcast it back with P_x. This gives the reconstructed label field $\widetilde{F}_x^{\mathrm{gt}}$, with the tilde marking the round trip drawn in the ground-truth pyramid of Figure 5. With pixel labels given, both transformations learn to preserve their motion. Writing $\mathcal{D}$ for the aggregated robust flow error, we minimize

$$\mathcal{L}_{\mathrm{flow}}(P_q, P_x \mid F_x^{\mathrm{gt}}) = \mathcal{D}\big(P_x P_q F_x^{\mathrm{gt}}, F_x^{\mathrm{gt}}\big)\,. \tag{6}$$

Both operators use trainable assignments in this round trip. Its reconstruction error directly teaches the grouping which motion differences to retain [20].

Joint training. Assignments and flow predictions are recomputed for each input pair, and all three objectives are optimized jointly. The unit and assignment terms supervise each refinement step, while reconstruction is added once. Together, they train shared features, unit predictions and assignments

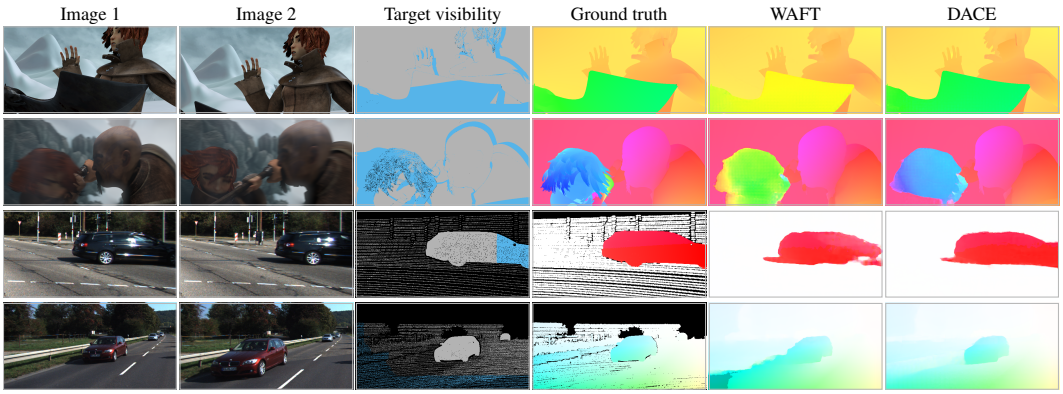


Figure 6: **DACE improves motion recovery and boundary precision.** Zero-shot examples show an occluded cape and blurred hair in Sintel (rows 1–2), and departing and approaching vehicles in KITTI (rows 3–4). All panels within a row share one crop; row 3 retains the original right image boundary. Blue marks occluded/out-of-frame pixels, gray other valid pixels, and black excluded pixels. Flow colors match within each pair; sparse KITTI labels are enlarged only for display.

to connect what a unit learns to match with the pixel motion it represents. Appendix G specifies the componentwise likelihoods, candidate weighting, validity handling, reductions and gradient routes.

4 Experiments

Our experiments test whether coupling a correspondence unit's matching evidence with the pixel motion it represents improves dense flow prediction. We examine zero-shot performance and regional gains, then use fixed-grid comparisons, ablations and computation profiles to assess the formulation and its design choices.

4.1 Experimental setup

Training and comparisons. All comparisons use two frames. We train DACE with a DINOv3 ViT-S backbone on FlyingChairs followed by FlyingThings3D. Things test results are in distribution; Sintel and KITTI training-split results are zero-shot, without target-domain flow training. Things and Sintel have Clean and Final renderings (C/F). We use WAFT's iterative framework [1], training with five updates and using five at inference unless stated otherwise. Both systems extract dense features on an $(H/2) \times (W/2)$ grid for an $H \times W$ image, abbreviated $H/2$. Released WAFT estimates flow on this fixed grid, whereas DACE uses adaptive units anchored at $H/4$, with a quarter as many units. We retain published results for the broad comparison and use the released WAFT checkpoint as the fine-grid reference.

Metrics. We report endpoint error (EPE), the Euclidean flow error in pixels averaged within each image and then over images, and KITTI Fl, the percentage of valid pixels whose error exceeds both 3 pixels and 5% of the ground-truth motion. Regional EPE pools errors over valid pixels in each specified region. Table 2 defines the boundary neighborhoods; Appendix B gives the masks and aggregation details. Computation is measured in billions of multiply–accumulate operations (GMACs) for one 540×960 image pair.

4.2 Zero-shot transfer and regional accuracy

Adaptive correspondence transfers to unseen scenes. Dense flow labels are difficult to acquire in real scenes [30, 31]. Zero-shot evaluation tests whether models generalize without target-domain flow supervision. DACE achieves the lowest Sintel Final and KITTI EPE among the compared methods (Table 1), with gains spanning rendered scenes and real driving imagery. With Depth Anything V2 [32], DACE again achieves lower EPE than the fine-grid reference (Appendix A).

Overall scores understate the gains where direct matches are missing. Occluded or out-of-frame pixels occupy only 6.4–13.9% of valid pixels yet contribute 42.3–52.2% of the fine-grid reference's

Table 1: **DACE achieves the lowest Sintel Final and KITTI EPE among the compared methods.** Zero-shot results on the Sintel and KITTI training splits after training on FlyingChairs and FlyingThings3D. Bold: lowest error in each column. –: not reported.

Method	Sintel		KITTI		MACs
	Clean	Final	EPE	Fl (%)	(G)
FlowNet2 [22]	2.02	3.14	10.10	30.4	230
PWC-Net [7]	2.55	3.93	10.40	33.7	101
RAFT [5]	1.43	2.71	5.04	17.4	938
GMA [10]	1.30	2.74	4.69	17.1	1352
SKFlow [11]	1.22	2.46	4.27	15.5	751
DIP [23]	1.30	2.82	4.29	13.7	3068
CRAFT [24]	1.27	2.79	4.88	17.5	2274
GMFlowNet [25]	1.14	2.71	4.24	15.4	1094
FlowFormer [9]	1.01	2.40	4.09	14.7	1715
EMD-L [26]	0.88	2.55	4.12	13.5	1755
FlowFormer++ [27]	0.90	2.30	3.93	14.1	1713
MatchFlow(G) [12]	1.03	2.45	4.08	15.6	1669
AnyFlow [15]	1.10	2.52	3.76	12.4	–
RPKNet [28]	1.12	2.45	–	13.0	137
CCMR+ [29]	0.98	2.36	–	12.9	12653
SEA-RAFT(L) [6]	1.19	4.11	3.62	12.9	655
SAMFlow [13]	0.87	2.11	3.44	12.3	9717
FlowDiffuser [14]	**0.86**	2.19	3.61	11.8	2466
DPFlow [16]	1.02	2.26	3.37	**11.1**	414
WAFT [1]	1.28	2.56	3.49	12.9	732
DACE	1.03	**2.10**	**3.11**	11.9	796

Table 2: **DACE lowers EPE where no match is visible and near motion boundaries.** Regional EPE pools errors over valid pixels. In (b), Pixel share is the percentage of valid pixels; Error share is their contribution to WAFT's standard EPE, retaining its image weighting. Things uses out-of-frame targets only. For (c), we define valid pixels in 5×5 neighborhoods of ground-truth motion boundaries using a 2-pixel local flow-change threshold (Appendix B.2); this region can overlap (a) and (b).

	(a) Visible		(b) Occluded/out-of-frame				(c) Near motion boundaries	
Dataset	WAFT	DACE	Pixel share (%)	Error share (%)	WAFT	DACE	WAFT	DACE
Things C	2.14	**1.37**	6.4	43.0	23.03	**16.61**	13.40	**9.24**
Things F	2.09	**1.40**	6.4	42.7	21.96	**15.89**	13.14	**9.48**
Sintel C	0.65	**0.49**	8.4	52.2	7.95	**6.77**	7.30	**6.63**
Sintel F	1.59	**1.28**	8.4	42.3	12.86	**10.93**	11.92	**10.45**
KITTI	**2.01**	2.05	13.9	51.9	13.65	**11.05**	9.97	**8.51**

standard EPE (Table 2(b)). DACE's absolute EPE reduction in these regions exceeds the overall reduction on every evaluated dataset and rendering. On KITTI, for example, occluded/out-of-frame EPE falls from 13.65 to 11.05, whereas overall EPE falls from 3.49 to 3.11. Visible-pixel EPE also improves on Things and Sintel and remains similar on KITTI (2.01 versus 2.05). The small share of affected pixels dilutes these larger gains in the full-image average. These results suggest greater overall benefit in similar scenes with more occluded or out-of-frame pixels.

Adaptive units improve accuracy near motion boundaries. We evaluate EPE near ground-truth motion boundaries to test whether the learned units preserve distinctions between neighboring motions. Sharing an estimate within a region of nearly uniform flow changes the result little; across a boundary, it can give a pixel the motion of the wrong surface. DACE improves boundary-region EPE on every evaluated dataset and rendering, despite using a quarter as many units as the fine-grid reference. These absolute reductions also exceed the overall reductions. The advantage persists at additional thresholds and against the $H/4$ control with local attention (Appendix B.2). The motion-boundary panels of Figure 4 illustrate how learned support follows a motion boundary.

Table 3: **Adaptive units improve flow accuracy and retain dense motion at the same unit count.** The systems share a backbone, feature resolution and training recipe. (a) Overall accuracy. (b) Occluded/out-of-frame EPE; attention gain is without minus with local attention. (c) Image-mean EPE after pooling ground-truth flow into $H/4$ units and applying each model's image-predicted readout, without retraining.

(a) Overall accuracy

		Things		Sintel		KITTI	
System	Attn.	C	F	C	F	EPE	Fl (%)
WAFT-H/4	–	2.93	2.72	1.08	2.49	3.96	13.3
WAFT-H/4	✓	2.73	2.62	1.07	2.57	3.75	12.6
DACE	–	2.56	2.48	1.12	2.14	3.54	13.0
DACE	✓	**2.39**	**2.38**	**1.03**	**2.10**	**3.11**	**11.9**

(c) Supplied-motion reconstruction EPE

WAFT-H/4	✓	0.50	0.53	0.21	0.39	1.52	—
DACE	✓	**0.19**	**0.21**	**0.09**	**0.19**	**0.39**	—

(b) Occluded/out-of-frame EPE

		Things		Sintel		KITTI
System	Attn.	C	F	C	F	EPE
WAFT-H/4	–	19.65	17.79	7.15	12.91	14.87
WAFT-H/4	✓	18.98	17.64	7.19	12.90	14.09
Attention gain		0.67	0.15	−0.04	0.01	0.78
DACE	–	17.49	16.55	6.89	11.40	12.96
DACE	✓	16.61	15.89	6.77	10.93	11.05
Attention gain		**0.88**	**0.66**	**0.12**	**0.47**	**1.91**

Qualitative comparison. Figure 6 shows improved recovery of the occluded cape and blurred hair on Sintel, and clearer separation of vehicle motion from the road on KITTI. Appendix C adds complementary comparisons and learned-support views.

4.3 Adaptive versus fixed-grid correspondence

We compare adaptive and fixed-grid systems at the same $H/4$ unit count. They share the backbone, dense-feature resolution, iterative framework, update count and training recipe. The fixed-grid controls use WAFT's code and loss; both formulations are evaluated with and without local attention. We examine accuracy, the use of context and the retention of pixel motion.

Adaptive units improve every reported accuracy result. DACE outperforms both fixed-grid controls on every reported accuracy result, including the control with the same local attention (Table 3(a)). When neither system uses local attention, DACE improves every reported result except Sintel Clean EPE.

Adaptive units benefit more from local context. Local attention reduces occluded/out-of-frame EPE more for adaptive units than for the fixed-grid control on every evaluated dataset and rendering (Table 3(b)).

Learned support retains pixel motion more faithfully. To test motion retention separately from correspondence accuracy, we replace predicted unit flow with pooled ground truth while keeping each model's image-predicted assignments and readout weights. Reconstruction EPE measures the motion detail lost through support and readout (Appendix D). DACE improves every reported reconstruction result (Table 3(c)); on Sintel Final, EPE falls from 0.39 to 0.19, and on KITTI from 1.52 to 0.39. DACE's learned support and readout preserve more motion detail at the same unit count.

4.4 Learning effective correspondence units

We vary one design choice at a time to test the effects of motion supervision, local context and spatial structure (Table 4).

Flow reconstruction teaches the grouping to preserve motion. Adding flow-reconstruction supervision improves every reported accuracy result. It directly trains assignments to preserve motion detail through pooling and broadcasting, complementing the unit and assignment losses.

Local attention strengthens each unit's matching evidence. Adding local attention improves every reported accuracy result. It provides evidence beyond a unit's pooled features while the assignments still define which pixels receive its flow. For pixels without direct matches, this attention also benefits DACE more than the fixed-grid control (Section 4.3).

Table 4: **Motion supervision, local context and spatial structure improve accuracy.** Each variant changes one choice from DACE. Prediction detachment blocks the assignment loss's gradients to unit flow and uncertainty; the pooled unit target remains detached in every variant.

					Things		Sintel		KITTI	
Variant	Flow recon.	Local attn.	Prediction detachment	Assignment scores	C	F	C	F	EPE	Fl (%)
DACE	✓	✓	✓	Gaussian	**2.39**	**2.38**	1.03	**2.10**	**3.11**	**11.9**
No flow reconstruction	–	✓	✓	Gaussian	2.68	2.61	1.05	2.17	3.14	12.4
No local attention	✓	–	✓	Gaussian	2.56	2.48	1.12	2.14	3.54	13.0
No prediction detachment	✓	✓	–	Gaussian	2.49	2.46	**1.00**	2.18	3.64	12.3
Direct logits	✓	✓	✓	Direct	2.58	2.55	1.02	2.24	3.16	12.1

Table 5: **Fewer units leave room for richer processing.** Costs are GMACs for one 540×960 image pair; errors are EPE except KITTI Fl. †: profiled only, not trained.

			Cost (GMACs)			Things		Sintel		KITTI	
System	Resolution	Updates	Attention	Refinement	Total	C	F	C	F	EPE	Fl (%)
WAFT-64 [1]	$H/2$	5	–	425.8	732.4	3.54	3.44	1.28	2.56	3.49	12.9
WAFT-128 + attn.†	$H/2$	5	57.0	943.9	1313.2	–	–	–	–	–	–
DACE-64	$H/4$	5	20.6	**293.8**	**636.3**	2.55	2.48	**1.03**	2.24	3.52	12.5
DACE-128	$H/4$	4	20.6	363.1	706.1	2.44	2.43	**1.03**	2.15	3.18	12.2
DACE-128 (main)	$H/4$	5	20.6	453.9	796.8	**2.39**	**2.38**	**1.03**	**2.10**	**3.11**	**11.9**

Separating direct supervision improves joint learning. Holding unit-flow predictions and uncertainty fixed for assignment supervision improves every reported result except Sintel Clean EPE. The pooled unit targets remain fixed in both variants. Unit predictions learn from pooled flow targets, and assignments learn which current predictions fit each pixel's motion. The features and assignments remain jointly learned.

Gaussian scores provide a useful spatial prior. Gaussian assignment scores improve every reported result except Sintel Clean EPE compared with directly predicted logits. The Gaussian constrains each pixel's assignment scores according to the relative positions of neighboring unit anchors (Eq. (9)). This spatial prior guides grouping while allowing irregular unit shapes.

4.5 Accuracy and computation

DACE predicts assignments at image resolution and performs matching and refinement over fewer units. Table 5 measures how this allocation affects accuracy and cost.

Fewer correspondence units allow richer processing. Suffixes denote recurrent-state and refinement width: WAFT-64 is the released reference, DACE-128 is our main model, and DACE-64 uses a narrower state. Widening the fine-grid baseline more than doubles its refinement cost; this configuration is profiled only. Local attention of the same width costs 20.6 GMACs on the adaptive lattice versus 57.0 on the fine grid. Fewer units make wider refinement and local attention more affordable.

Four-update DACE outperforms five-update WAFT at lower cost. With four updates, DACE outperforms five-update WAFT on every reported accuracy result while using 706.1 versus 732.4 GMACs. Our five-update default uses 796.8 GMACs. The narrower DACE-64 costs 636.3 GMACs and improves every reported result except KITTI EPE. Appendix E gives the full refinement sweep.

5 Conclusion

DACE learns the units of correspondence for dense optical flow. Shared pixel-to-unit assignments pool matching features and distribute unit flow to pixels. The assignments and matching features are

learned jointly from flow supervision. Motion is estimated on coarser units, while image evidence stays dense. DACE achieves the lowest zero-shot Sintel Final and KITTI EPE among the compared published methods, with fewer MACs than most. Even against a fixed grid with four times as many units, DACE lowers EPE on every dataset, is more robust where no match is visible, and is more precise near motion boundaries. At the same unit count, DACE outperforms the fixed-grid controls in every reported accuracy comparison. Adaptive units improve both the use of local context and the preservation of pixel motion.

Acknowledgements

This project was supported in part by NSF grant 2313151, with additional computing resources provided through NAIRR Pilot CIS240421 and CIS240431.

References

[1] Yihan Wang and Jia Deng. WAFT: Warping-Alone Field Transforms for Optical Flow, October 2025.

[2] Bruce D Lucas and Takeo Kanade. An Iterative Image Registration Technique with an Application to Stereo Vision. In *IJCAI'81: 7th International Joint Conference on Artificial Intelligence*, volume 2, pages 674–679, Vancouver, Canada, August 1981.

[3] Berthold K. P. Horn and Brian G. Schunck. Determining optical flow. *Artificial Intelligence*, 17 (1):185–203, August 1981. ISSN 0004-3702. doi: 10.1016/0004-3702(81)90024-2.

[4] Michael J. Black and P. Anandan. The robust estimation of multiple motions: Parametric and piecewise-smooth flow fields. *Computer Vision and Image Understanding*, 63(1):75–104, 1996. URL `https://www.csd.uoc.gr/~hy576/bibliography/RobustMotion.pdf`.

[5] Zachary Teed and Jia Deng. RAFT: Recurrent All-Pairs Field Transforms for Optical Flow. In Andrea Vedaldi, Horst Bischof, Thomas Brox, and Jan-Michael Frahm, editors, *Computer Vision – ECCV 2020*, pages 402–419, Cham, 2020. Springer International Publishing. ISBN 978-3-030-58536-5. doi: 10.1007/978-3-030-58536-5_24.

[6] Yihan Wang, Lahav Lipson, and Jia Deng. SEA-RAFT: Simple, Efficient, Accurate RAFT for Optical Flow. In Aleš Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol, editors, *Computer Vision – ECCV 2024*, pages 36–54, Cham, 2025. Springer Nature Switzerland. ISBN 978-3-031-72667-5. doi: 10.1007/978-3-031-72667-5_3.

[7] Deqing Sun, Xiaodong Yang, Ming-Yu Liu, and Jan Kautz. PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 8934–8943, 2018.

[8] Haofei Xu, Jing Zhang, Jianfei Cai, Hamid Rezatofighi, and Dacheng Tao. GMFlow: Learning Optical Flow via Global Matching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8121–8130, 2022.

[9] Zhaoyang Huang, Xiaoyu Shi, Chao Zhang, Qiang Wang, Ka Chun Cheung, Hongwei Qin, Jifeng Dai, and Hongsheng Li. FlowFormer: A Transformer Architecture for Optical Flow. In Shai Avidan, Gabriel Brostow, Moustapha Cissé, Giovanni Maria Farinella, and Tal Hassner, editors, *Computer Vision – ECCV 2022*, pages 668–685, Cham, 2022. Springer Nature Switzerland. ISBN 978-3-031-19790-1. doi: 10.1007/978-3-031-19790-1_40.

[10] Shihao Jiang, Dylan Campbell, Yao Lu, Hongdong Li, and Richard Hartley. Learning To Estimate Hidden Motions With Global Motion Aggregation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9772–9781, 2021.

[11] Shangkun Sun, Yuanqi Chen, Yu Zhu, Guodong Guo, and Ge Li. SKFlow: Learning Optical Flow with Super Kernels. In *Advances in Neural Information Processing Systems*, 2022.

[12] Qiaole Dong, Chenjie Cao, and Yanwei Fu. Rethinking Optical Flow from Geometric Matching Consistent Perspective. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1337–1347, Vancouver, BC, Canada, June 2023. IEEE. ISBN 979-8-3503-0129-8. doi: 10.1109/CVPR52729.2023.00135.

[13] Shili Zhou, Ruian He, Weimin Tan, and Bo Yan. SAMFlow: Eliminating Any Fragmentation in Optical Flow with Segment Anything Model. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(7):7695–7703, March 2024. ISSN 2374-3468. doi: 10.1609/aaai.v38i7.28603.

[14] Ao Luo, Xin Li, Fan Yang, Jiangyu Liu, Haoqiang Fan, and Shuaicheng Liu. FlowDiffuser: Advancing Optical Flow Estimation with Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19167–19176, 2024.

[15] Hyunyoung Jung, Zhuo Hui, Lei Luo, Haitao Yang, Feng Liu, Sungjoo Yoo, Rakesh Ranjan, and Denis Demandolx. AnyFlow: Arbitrary Scale Optical Flow With Implicit Neural Representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5455–5465, 2023.

[16] Henrique Morimitsu, Xiaobin Zhu, Roberto M. Cesar, Xiangyang Ji, and Xu-Cheng Yin. DPFlow: Adaptive Optical Flow Estimation with a Dual-Pyramid Framework. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17810–17820, 2025.

[17] Xiaofeng Ren. Local grouping for optical flow. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 1–8, 2008. URL `https://iacl.ece.jhu.edu/proceedings/cvpr2008/papers/196.pdf`.

[18] Radhakrishna Achanta, Appu Shaji, Kevin Smith, Aurelien Lucchi, Pascal Fua, and Sabine Süsstrunk. SLIC Superpixels Compared to State-of-the-Art Superpixel Methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 34(11):2274–2282, November 2012. ISSN 1939-3539. doi: 10.1109/TPAMI.2012.120.

[19] Michael Van den Bergh, Xavier Boix, Gemma Roig, Benjamin de Capitani, and Luc Van Gool. SEEDS: Superpixels Extracted via Energy-Driven Sampling. In Andrew Fitzgibbon, Svetlana Lazebnik, Pietro Perona, Yoichi Sato, and Cordelia Schmid, editors, *Computer Vision – ECCV 2012*, pages 13–26, Berlin, Heidelberg, 2012. Springer. ISBN 978-3-642-33786-4. doi: 10.1007/978-3-642-33786-4_2.

[20] Varun Jampani, Deqing Sun, Ming-Yu Liu, Ming-Hsuan Yang, and Jan Kautz. Superpixel Sampling Networks, July 2018.

[21] Fengting Yang, Qian Sun, Hailin Jin, and Zihan Zhou. Superpixel Segmentation With Fully Convolutional Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13964–13973, 2020.

[22] Eddy Ilg, Nikolaus Mayer, Tonmoy Saikia, Margret Keuper, Alexey Dosovitskiy, and Thomas Brox. FlowNet 2.0: Evolution of Optical Flow Estimation With Deep Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2462–2470, 2017.

[23] Zihua Zheng, Ni Nie, Zhi Ling, Pengfei Xiong, Jiangyu Liu, Hao Wang, and Jiankun Li. DIP: Deep Inverse Patchmatch for High-Resolution Optical Flow. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8925–8934, 2022.

[24] Xiuchao Sui, Shaohua Li, Xue Geng, Yan Wu, Xinxing Xu, Yong Liu, Rick Goh, and Hongyuan Zhu. CRAFT: Cross-Attentional Flow Transformer for Robust Optical Flow. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17602–17611, 2022.

[25] Shiyu Zhao, Long Zhao, Zhixing Zhang, Enyu Zhou, and Dimitris Metaxas. Global Matching with Overlapping Attention for Optical Flow Estimation. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17571–17580, New Orleans, LA, USA, June 2022. IEEE. ISBN 978-1-6654-6946-3. doi: 10.1109/CVPR52688.2022.01707.

[26] Changxing Deng, Ao Luo, Haibin Huang, Shaodan Ma, Jiangyu Liu, and Shuaicheng Liu. Explicit Motion Disentangling for Efficient Optical Flow Estimation. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9487–9496, Paris, France, October 2023. IEEE. ISBN 979-8-3503-0718-4. doi: 10.1109/ICCV51070.2023.00873.

[27] Xiaoyu Shi, Zhaoyang Huang, Dasong Li, Manyuan Zhang, Ka Chun Cheung, Simon See, Hongwei Qin, Jifeng Dai, and Hongsheng Li. FlowFormer++: Masked Cost Volume Autoencoding for Pretraining Optical Flow Estimation. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1599–1610, Vancouver, BC, Canada, June 2023. IEEE. ISBN 979-8-3503-0129-8. doi: 10.1109/CVPR52729.2023.00160.

[28] Henrique Morimitsu, Xiaobin Zhu, Xiangyang Ji, and Xu-Cheng Yin. Recurrent Partial Kernel Network for Efficient Optical Flow Estimation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(5):4278–4286, March 2024. ISSN 2374-3468. doi: 10.1609/aaai.v38i5.28224.

[29] Azin Jahedi, Maximilian Luz, Marc Rivinius, and Andrés Bruhn. CCMR: High Resolution Optical Flow Estimation via Coarse-To-Fine Context-Guided Motion Reasoning. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 6899–6908, 2024.

[30] Daniel J. Butler, Jonas Wulff, Garrett B. Stanley, and Michael J. Black. A Naturalistic Open Source Movie for Optical Flow Evaluation. In Andrew Fitzgibbon, Svetlana Lazebnik, Pietro Perona, Yoichi Sato, and Cordelia Schmid, editors, *Computer Vision – ECCV 2012*, pages 611–625, Berlin, Heidelberg, 2012. Springer. ISBN 978-3-642-33783-3. doi: 10.1007/978-3-642-33783-3_44.

[31] Moritz Menze and Andreas Geiger. Object Scene Flow for Autonomous Vehicles. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3061–3070, 2015.

[32] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth Anything V2. *Advances in Neural Information Processing Systems*, 37:21875–21911, December 2024. doi: 10.52202/079017-0688.

[33] MPI Sintel. Optical flow evaluation metrics. `https://sintel.is.tuebingen.mpg.de/results`. Accessed September 29, 2026.

Appendix overview. We examine whether the accuracy gains extend across backbones (Appendix A), define the regional evaluations and test their robustness (Appendix B), and provide additional flow and learned-support visualizations (Appendix C). Appendix D details the supplied-motion reconstruction study, Appendix E extends the accuracy–computation comparison, and Appendix F reports the limits of the current results after target-specific fine-tuning. Appendix G specifies the assignment parameterization and training objectives.

Common setting. Unless stated otherwise, we use the DINOv3 systems from Section 4.1: DACE, released WAFT, and the fixed-grid WAFT-H/4 control with local attention. Models are trained on FlyingChairs and FlyingThings3D; Things test results are in distribution, while Sintel and KITTI training-split results are zero-shot. Backbone and fine-tuning comparisons use the separately identified settings below.

A Zero-Shot Transfer across Backbones

DACE lowers zero-shot EPE on both Sintel renderings and KITTI with DINOv3 and Depth Anything V2 (DAv2) [32] (Table 6). All models use Chairs–Things flow training; WAFT scores are its published results [1]. KITTI Fl also improves with DINOv3 and ties at the reported precision with DAv2.

Table 6: **DACE lowers EPE with both backbones.** Zero-shot results on the Sintel and KITTI training splits after training on FlyingChairs and FlyingThings3D. Bold: better of each pair.

		Sintel		KITTI	
Backbone	Method	Clean	Final	EPE	Fl (%)
DINOv3	WAFT [1]	1.28	2.56	3.49	12.9
	DACE	**1.03**	**2.10**	**3.11**	**11.9**
DAv2	WAFT [1]	1.01	2.49	3.28	**10.9**
	DACE	**0.97**	**2.30**	**2.93**	**10.9**

B Regional Evaluation and Robustness

The regional analyses test motion estimation without direct target observations and the separation of neighboring motions. We define the pixel sets used in Section 4.2, explain their contribution to overall error, and examine additional boundary thresholds and large displacements. Regional EPE pools errors over the valid pixels in each specified set; boundary regions can overlap either visibility category.

B.1 Occluded and out-of-frame pixels

Pixel sets. A pixel is out of frame if its ground-truth endpoint $x + F_x^{\mathrm{gt}}(x)$ falls outside Image 2. On Sintel and KITTI, we combine these pixels with those whose targets are marked occluded by the dataset annotations. On Things, we use endpoint bounds only, so this set contains out-of-frame pixels. All sets are restricted to the valid pixels used for standard EPE.

Sintel's official invalid mask further excludes pixels from both visibility categories. These pixels can remain in the standard-EPE valid set, so the two categories need not exhaust that set. Pixel coverage always uses the full standard-EPE valid set as its denominator.

Contribution to overall error. Occluded or out-of-frame pixels occupy 6.4–13.9% of valid pixels but account for 42.3–52.2% of WAFT's standard EPE (Table 2(b)). This concentration shows why the overall score alone does not explain where substantial errors remain, even with fine-grid correspondence.

To measure this contribution, we retain the image weighting used by standard EPE. For K image pairs, let V_i be the standard-EPE valid set of pair i, $O_i \subseteq V_i$ its occluded/out-of-frame set, and $e_i(x) = \|F_{x,i}(x) - F_{x,i}^{\mathrm{gt}}(x)\|_2$ the endpoint error. If E is the standard EPE over these pairs, the

fraction contributed by this set is

$$B_O = \frac{1}{KE} \sum_{i=1}^{K} \frac{\sum_{x \in O_i} e_i(x)}{|V_i|}. \tag{7}$$

Each inner term is the region's contribution to that image's mean error; averaging these contributions and dividing by E gives the fraction B_O. Table 2(b) reports this fraction as a percentage, $100B_O$. The numerator and denominator use the same evaluation pass. This image-weighted error share differs from regional EPE, which pools errors over pixels.

B.2 Accuracy near motion boundaries

Defining the evaluation region. To test accuracy where neighboring motions differ, we measure EPE over a region we define around ground-truth motion boundaries. We identify these boundaries from local changes in ground-truth flow F_x^{gt}, with strength

$$s(x) = \|\Delta_1 F_x^{\mathrm{gt}}(x)\|_2 + \|\Delta_2 F_x^{\mathrm{gt}}(x)\|_2, \tag{8}$$

where each Δ takes the difference to the next pixel along one image axis. A difference contributes only when both pixels have valid labels. For threshold τ, pixels with $s(x) > \tau$ seed the boundary region, which contains valid pixels in a 5×5 window around any seed. We pool endpoint errors over this region. We evaluate $\tau = 2, 3, 4$ pixels; the main paper uses $\tau = 2$. At this threshold, the region covers 9.3% of valid pixels on Things, 6.2% on Sintel and 13.6% on KITTI.

Boundary precision improves across the tested thresholds. DACE has lower EPE in these neighborhoods than both fixed-grid references on every dataset and rendering at all three thresholds (Table 7). Released WAFT uses four times as many correspondence units, while WAFT-H/4 with local attention uses the same number as DACE.

Table 7: **DACE has lower EPE near motion boundaries at all tested thresholds.** EPE is pooled over a region we define around ground-truth motion boundaries: valid pixels in 5×5 windows around boundary seeds. The threshold sets how large a local flow change must be to count as a boundary. WAFT estimates motion for four times as many units as DACE, and WAFT-H/4 with attention for the same number.

	Things		Sintel		KITTI
System	Clean	Final	Clean	Final	EPE
(a) Boundary threshold: 2 pixels					
WAFT	13.40	13.14	7.30	11.92	9.97
WAFT-H/4 + attention	10.77	10.65	6.97	11.40	11.75
DACE	**9.24**	**9.48**	**6.63**	**10.45**	**8.51**
(b) Boundary threshold: 3 pixels					
WAFT	12.50	12.37	7.38	12.52	12.84
WAFT-H/4 + attention	10.08	10.07	7.02	11.90	15.30
DACE	**8.56**	**8.77**	**6.64**	**11.03**	**10.65**
(c) Boundary threshold: 4 pixels					
WAFT	12.29	12.26	7.75	13.38	15.75
WAFT-H/4 + attention	10.02	9.99	7.34	12.73	18.87
DACE	**8.40**	**8.63**	**6.85**	**11.78**	**12.76**

B.3 Missing targets with large displacements

We next test whether the missing-target improvement persists for pixels with large motion. Following the 40-pixel cutoff in Sintel's $s40+$ category [33], we intersect each occluded/out-of-frame set with pixels whose ground-truth flow magnitude exceeds 40 pixels, using the same cutoff on all datasets.

DACE lowers EPE relative to both fixed-grid references on every evaluated dataset and rendering in this subset (Table 8).

Table 8: **DACE improves occluded/out-of-frame flow even at large displacements.** EPE pooled over pixels whose target is occluded or out of frame and whose ground-truth displacement exceeds 40 pixels. Pixel share: this subset's percentage of valid pixels.

	Things		Sintel		KITTI
System	Clean	Final	Clean	Final	EPE
WAFT	33.03	31.41	19.19	29.00	15.34
WAFT-H/4 + attention	27.19	25.25	17.25	27.48	15.81
DACE	**23.77**	**22.73**	**16.16**	**24.08**	**12.36**
Pixel share (%)	4.4	4.4	2.7	2.7	12.2

C Qualitative Flow and Learned Support

These examples connect flow comparisons with learned assignments in the same image regions, using the zero-shot DINOv3 models from Figure 6. Figure 7 adds assignment details for the main paper's departing vehicle. Figure 8 adds two comparisons with clear improvements over WAFT, spanning crossing structures and a broad moving surface.

Reading the comparisons. Each example uses one contextual crop with the same coordinates in both input frames, the visibility map, ground truth and both predictions. All panels share a 2:1 aspect ratio. The input pair establishes movement; the aligned flow maps show the resulting motion. These selected examples illustrate the improvements; the quantitative tables assess performance across the datasets.

Reading the assignments. The box in Image 1 locates the enlarged image, learned-support and assignment-entropy details. White contours separate pixels whose strongest assignments select different units; the model itself uses soft weights. Assignment entropy, normalized by $\log 9$, ranges from dark blue (weight concentrated on one unit) to yellow (weight spread across units). It describes assignment spread, not flow confidence. These maps let the reader inspect the grouping; accuracy is assessed from flow comparisons and the quantitative results.

Visualization protocol. Ground truth and both predictions share the Middlebury color wheel within each pair. On Sintel, magnitude m uses radius $\log(1 + m)/6$; on KITTI, radius grows linearly up to the largest valid ground-truth magnitude. Black in ground-truth flow denotes missing labels. Sparse KITTI labels are enlarged only for display. In target-visibility maps, blue denotes occluded/out-of-frame pixels, gray other valid pixels, and black excluded pixels.

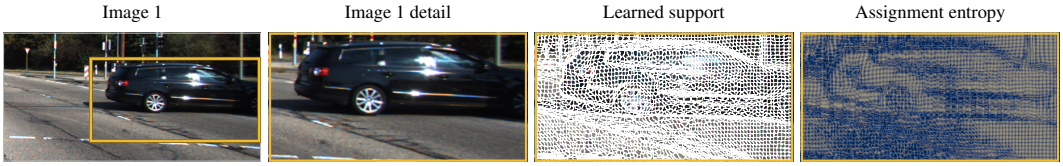


Figure 7: **Learned support for the departing vehicle.** The vehicle is the same as in Figure 6, row 3. The box in Image 1 locates the enlarged image, learned-support and assignment-entropy panels. Its flow comparison is shown only in the main paper.

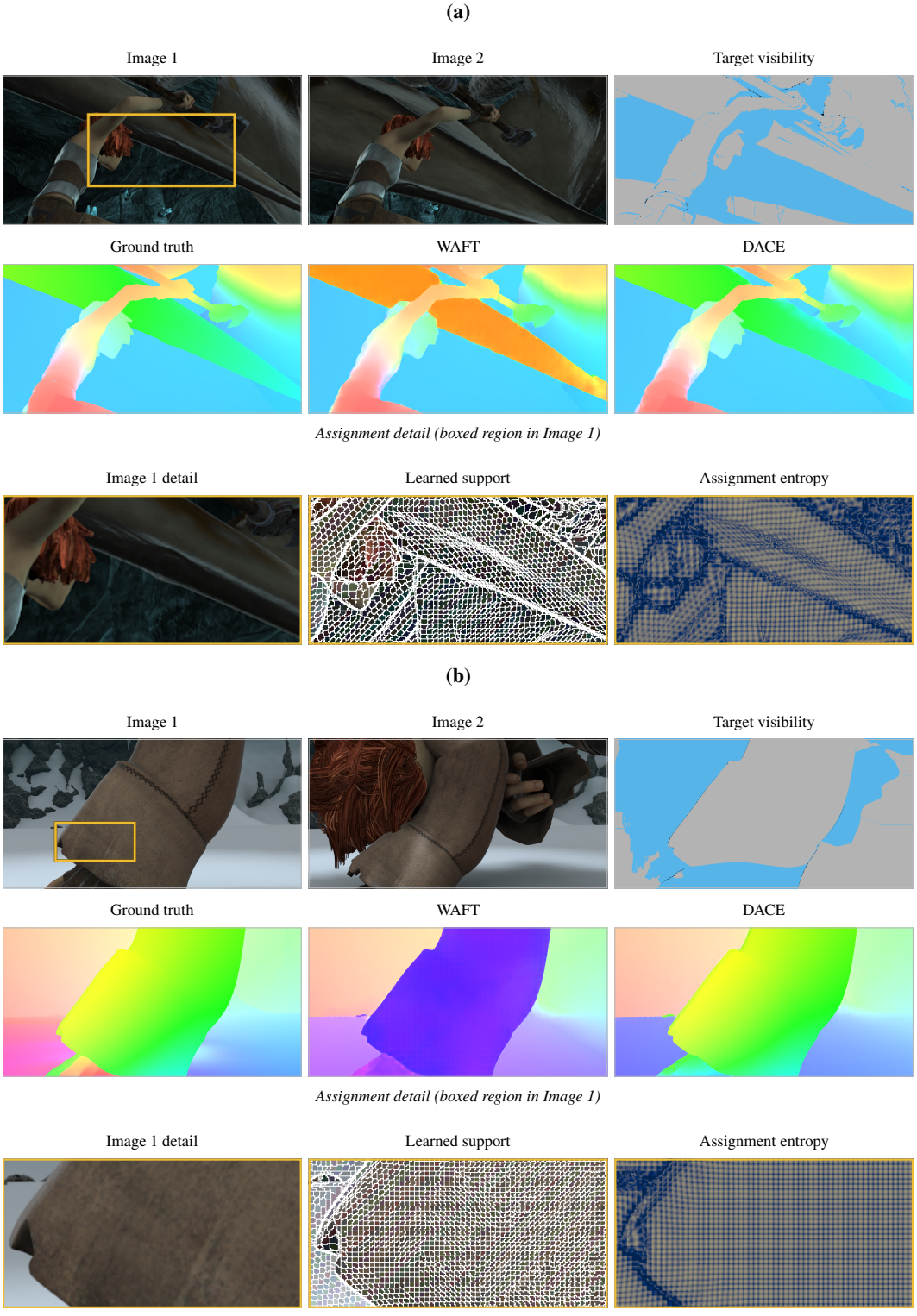


Figure 8: **DACE recovers motion on both thin and broad structures.** (a) WAFT assigns incorrect motion to the wing behind the crossing limbs; DACE recovers the wing's motion. (b) DACE recovers the sleeve's spatially varying motion. Both are Sintel Clean pairs. Gold-framed details show the learned support and assignment entropy.

D Supplied-Motion Reconstruction Protocol

This diagnostic measures the motion detail retained by each model's support and readout, separately from the accuracy of its unit-flow predictions. Table 3(c) in Section 4.3 reports the results.

Substituting pooled ground-truth flow. We first obtain each model's assignments and readout weights from the input images. For DACE, P_q pools ground-truth flow into units and P_x broadcasts it back through the same assignments A. This is the ground-truth round trip used by reconstruction supervision (Eq. (6)) and illustrated in Figure 5. For WAFT-H/4 with local attention, we average ground-truth flow within each fixed cell and upsample it with the predicted convex weights. These image-predicted weights remain fixed during the substitution; neither model is retrained. Pooling and evaluation use valid ground-truth pixels. We report reconstruction EPE, averaged within each image and then over images, rather than the robust training penalty $\mathcal{D}$ in Eq. (6).

What the reconstruction measures. Lower reconstruction EPE means that less motion detail is lost through support and readout. DACE preserves more detail than the fixed-grid control on every evaluated dataset and rendering at the same $H/4$ unit count. The comparison evaluates support and readout together; it does not isolate their effects or decompose the end-to-end prediction error.

E Accuracy and Computation across Refinement Steps

This sweep extends Section 4.5 by varying inference updates without retraining. Table 9 evaluates the same trained DACE and WAFT checkpoints with one to five updates. Both were trained with five updates, and their weights remain fixed during this sweep. The separately trained DACE-64 variant uses narrower recurrent-state and refinement features and five updates.

Computation is measured for one 540×960 image pair with model-specific padding. Counts include feature extraction, unit matching-feature construction, all refinement updates, and dense-flow readout.

Fewer updates retain a favorable accuracy–computation trade-off. Four-update DACE outperforms five-update WAFT in every reported accuracy comparison at 706.1 versus 732.4 GMACs. Relative to five-update WAFT, three-update DACE improves every reported result except KITTI Fl at 16% lower computation; DACE-64 improves every reported result except KITTI EPE at 13% lower computation.

At the same update count, DACE improves every reported accuracy result but uses more computation.

Table 9: **Four-update DACE outperforms five-update WAFT at lower cost.** Both models are trained with five updates and run here with one to five. DACE-64 halves the recurrent state width.

		Things		Sintel		KITTI		
System	Updates	Clean	Final	Clean	Final	EPE	Fl (%)	MACs (G)
WAFT	1	6.70	6.44	2.20	3.50	8.47	37.1	391.8
DACE	1	5.01	4.84	1.54	2.67	8.30	36.9	433.8
WAFT	2	4.27	4.15	1.49	2.88	4.67	18.5	477.0
DACE	2	2.91	2.84	1.10	2.29	4.09	17.1	524.5
WAFT	3	3.78	3.66	1.34	2.70	3.81	14.0	562.1
DACE	3	2.55	2.49	1.06	2.22	3.38	13.2	615.3
WAFT	4	3.61	3.51	1.29	2.56	3.56	13.1	647.3
DACE	4	2.44	2.43	1.03	2.15	3.18	12.2	706.1
WAFT	5	3.54	3.44	1.28	2.56	3.49	12.9	732.4
DACE	5	2.39	2.38	1.03	2.10	3.11	11.9	796.8
DACE-64	5	2.55	2.48	1.03	2.24	3.52	12.5	636.3

F Target-Specific Fine-Tuning

The zero-shot advantage does not carry over to the current fine-tuned models. Table 10 compares DACE, fine-tuned with WAFT's recipe and submitted to the benchmark servers, with WAFT's published DINOv3 results [1]. These results use the hidden test sets of Sintel, KITTI 2015, and Spring, with target-benchmark training rather than the standard zero-shot setting above.

DACE matches WAFT's reported Sintel Final EPE and trails it on Sintel Clean, both KITTI Fl variants, and all four Spring metrics. Extending the observed transfer gains to target-specific fine-tuning remains an open direction.

Benchmark metrics. Sintel reports EPE. KITTI reports Fl over all labeled pixels (Fl-all) and non-occluded pixels (Fl-noc). Spring provides four ground-truth values per image pixel and measures error against the closest one. Its 1px metric is the percentage of pixels with endpoint error above one pixel, and WAUC summarizes accuracy across error thresholds. Lower is better except for WAUC.

Table 10: **After fine-tuning, DACE matches WAFT on Sintel Final but trails it elsewhere.** Benchmark test results for DACE and DINOv3 WAFT. Lower is better except WAUC (↑).

	Sintel		KITTI 2015		Spring			
System	Clean	Final	Fl-all (%)	Fl-noc (%)	1px (%)	EPE	Fl (%)	WAUC ↑
WAFT	0.94	2.02	3.56	2.13	3.18	0.33	1.25	95.05
DACE	1.06	2.02	3.67	2.35	3.43	0.35	1.30	94.86

G Assignment Parameterization and Training Objectives

We specify the assignment scores and training objectives summarized in Section 3. The operators and flow fields retain the same pixel/unit subscripts. Matrix entries select an individual pixel or unit and flow component; for example, $[F_q]_{q,c}$ is component c of unit q's flow. Both transformations in the loss signatures are determined by the shared assignments A.

Gaussian assignment scores. For pixel x, the anchor of its lattice cell defines the local origin. Its candidate anchors $q \in \mathcal{N}(x)$ have offsets $\delta_{xq} \in \{-1, 0, 1\}^2$. The five predicted parameters are a mean offset $\mu_x \in \mathbb{R}^2$ and three entries of a lower-triangular factor L_x with positive diagonal, which defines the inverse covariance $G_x = L_x L_x^\top$. The scores and normalized assignments are

$$a_{xq} = -\tfrac{1}{2}(\delta_{xq} - \mu_x)^\top G_x (\delta_{xq} - \mu_x), \qquad A_{xq} = \frac{\exp a_{xq}}{\sum_{r \in \mathcal{N}(x)} \exp a_{xr}}. \tag{9}$$

Entries outside $\mathcal{N}(x)$ are zero. The parameterization couples each pixel's candidate scores through their relative spatial positions; it does not constrain a unit's collected support to a Gaussian shape.

Probabilistic unit-flow predictions. The probabilistic flow field $\mathcal{F}_q^{(t)}$ comprises a distribution for each flow component of each unit. We write its density as $p_{q,c}^{(t)}(f)$. Following SEA-RAFT [6], it is a two-component Laplace mixture centered on the corresponding flow estimate:

$$p_{q,c}^{(t)}(f) = \sum_{k=1}^{2} \pi_{q,k}^{(t)} \frac{1}{2b_{q,k}^{(t)}} \exp\left(-\frac{|f - [F_q^{(t)}]_{q,c}|}{b_{q,k}^{(t)}} \right). \tag{10}$$

Here $c \in \{1, 2\}$ indexes the flow components, and $\pi_{q,k}^{(t)}$ are softmax-normalized mixture weights. Both mixture components share the flow center. The implementation bounds the first log-scale to $[0, 10]$ and fixes the second to zero; scales and mixture weights are shared across the two flow components. These mixture weights are distinct from pixel-to-unit assignments. Image inputs and model parameters are suppressed in the notation.

Valid labels and pooled targets. Let V denote the valid training pixels, including the inherited flow-magnitude filter. Write $P_q^V(A)$ for weighted pooling restricted to valid labels. The unit target

is $F_q^{\mathrm{gt}} = P_q^V(A)F_x^{\mathrm{gt}}$, and the validity weight $w_q = [P_q(A)\mathbf{1}_V]_q$ is the fraction of a unit's support with valid labels. The targets and validity weights are recomputed from the current assignments and held fixed for unit supervision. Feature pooling and matching remain differentiable through the assignments.

Mixtures of neighboring flow predictions. The pixelwise mixture $\mathrm{Mix}_{P_x}[\mathcal{F}_q^{(t)}]$ in Section 3.3 has component density $p_{x,c}^{(t)}$. Since $[P_x]_{xq} = A_{xq}$, its weights come from the pixel assignments. The assignment objective retains the four largest weights for each pixel and renormalizes them over that set, $\mathcal{S}_4(x)$:

$$p_{x,c}^{(t)}(f) = \sum_{q \in \mathcal{S}_4(x)} \frac{A_{xq}}{\sum_{r \in \mathcal{S}_4(x)} A_{xr}} \, p_{q,c}^{(t)}(f). \tag{11}$$

Unit-flow and uncertainty predictions are held fixed for this objective; the assignments and their shared features learn through the mixture weights. The prediction-detachment ablation in Table 4 removes this restriction while keeping the pooled unit targets fixed. Mixing is performed separately for the two flow components, rather than forming a likelihood around an averaged flow vector. Dense inference still uses all nine assignment weights in $F_x = P_x F_q$.

Loss reductions and joint training. The NLL operator in Section 3.3 takes a predicted distribution first and a target second. It averages over flow components and aggregates over units or valid pixels. With pooled targets and validity weights fixed for unit supervision, and unit predictions fixed for assignment supervision, the two objectives are

$$\mathcal{L}_{\mathrm{unit}}^{(t)} = -\frac{1}{2\sum_q w_q} \sum_q w_q \sum_{c=1}^{2} \log p_{q,c}^{(t)}\big([F_q^{\mathrm{gt}}]_{q,c}\big)\,, \tag{12}$$

$$\mathcal{L}_{\mathrm{assign}}^{(t)} = -\frac{1}{2|V|} \sum_{x \in V} \sum_{c=1}^{2} \log p_{x,c}^{(t)}\big([F_x^{\mathrm{gt}}]_{x,c}\big)\,. \tag{13}$$

For motion preservation, we recompute $\widetilde{F}_x^{\mathrm{gt}} = P_x(A)P_q^V(A)F_x^{\mathrm{gt}}$ with both transformations differentiable. With $\rho(z) = \sqrt{\|z\|_2^2 + 10^{-6}} - 10^{-3}$, the reconstruction objective, specifying $\mathcal{D}$ in Eq. (6), is

$$\mathcal{L}_{\mathrm{flow}} = \frac{1}{|V|} \sum_{x \in V} \rho\Big([\widetilde{F}_x^{\mathrm{gt}}]_{x,:} - [F_x^{\mathrm{gt}}]_{x,:}\Big)\,. \tag{14}$$

This branch does not reuse the fixed unit-supervision target. Training numerators and denominators pool entries across the minibatch; these training reductions are distinct from benchmark evaluation conventions.

The first two objectives supervise every refinement step, while reconstruction is added once:

$$\mathcal{L} = \sum_{t=1}^{T} \gamma^{T-t} \left(\mathcal{L}_{\mathrm{unit}}^{(t)} + \mathcal{L}_{\mathrm{assign}}^{(t)}\right) + \lambda_{\mathrm{flow}} \mathcal{L}_{\mathrm{flow}}. \tag{15}$$

We use $T = 5$, $\gamma = 0.85$, and $\lambda_{\mathrm{flow}} = 1$. All objectives are optimized jointly. Holding the stated predictions fixed within one objective separates its direct gradient paths; it does not freeze the shared features or introduce alternating optimization.